
HỌC PHẦN NÂNG CAO VỚI C++

BỘ MÔN CÔNG NGHỆ THÔNG TIN

CHƯƠNG 2: LỚP VÀ ĐỐI TƯỢNG

BỘ MÔN CÔNG NGHỆ THÔNG TIN

- 2.1. Định nghĩa Lớp và các thành phần của lớp.
- 2.2. Phạm vi Private, Protected, Public
- 2.3. Đối tượng, mảng đối tượng.
- 2.4. Hàm tạo và hàm hủy.
- 2.5. Con trỏ this
- 2.6. Lớp bao
- 2.7. Hàm bạn
- 2.8. Lớp bạn
- 2.9. Nạp chồng

2.1. Định nghĩa Lớp và các thành phần của lớp.

- ❖ Khái niệm Lớp và các thành phần của lớp.
 - Thuộc tính
 - Phương thức
- ❖ Phân tích và xác định các thành phần của lớp.
- ❖ Định nghĩa và sử dụng lớp

2.1.1. Khái niệm lớp và các thành phần của lớp

- ❖ Lớp còn gọi là lớp đối tượng được dùng để mô tả tập các đối tượng cùng kiểu, trong khi đó mỗi đối tượng thuộc về một lớp cụ thể được dùng để chỉ một thực thể cụ thể của lớp nào đó.
- ❖ Phân tích và xác định thành phần của lớp dựa trên các đối tượng của lớp. Mọi đối tượng đều đặc trưng bởi hai thành phần
 - Dữ liệu (Thuộc tính và trạng thái)
 - Hành động (phương thức)

Ví dụ

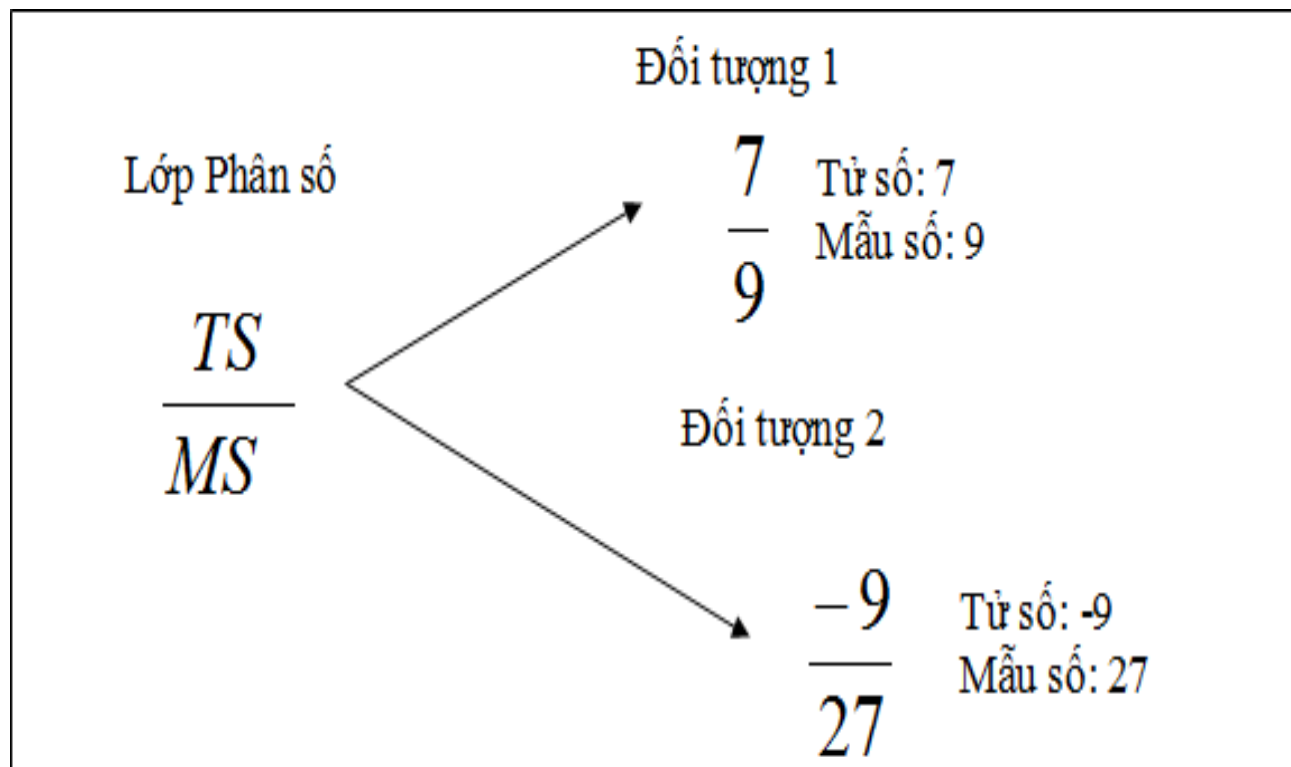
	Trạng thái	Hành động
Con mèo 	- Tên - Màu lông - Vui sướng	- Kêu - Vẫy đuôi - Chạy - Ăn - Bắt chuột
Xe đạp 	- Bánh răng - Bàn đạp - Dây xích - Bánh xe	- Tăng tốc - Giảm tốc - Chuyển bánh răng

2.1.2. Phân tích và xác định thành phần của lớp

- ❖ Xác định lớp ta cần xác định thành phần dữ liệu để lưu thông tin của đối tượng (thuộc tính); thành phần phương thức (hành động) là thành phần tác động lên dữ liệu và có thể làm thay đổi nội dung của dữ liệu.
- ❖ Để xác định thành phần thuộc tính và phương thức của lớp ta cần:
 - Nghiên cứu tập các đối tượng của lớp cần xác định
 - Liệt kê tập thuộc tính, các phương thức
 - Sử dụng phương pháp trừu tượng hóa để trích lọc thuộc tính và phương thức cần thiết.

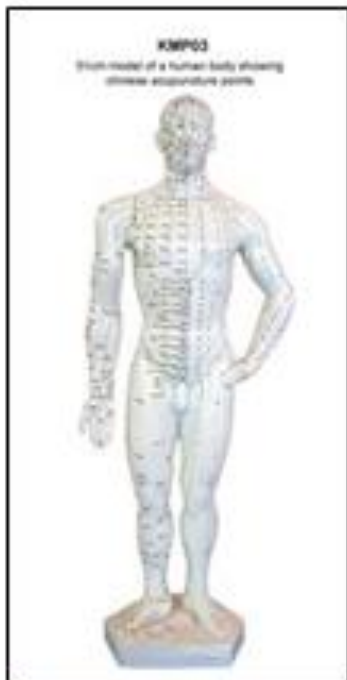
a. Xác định thành phần thuộc tính

- ❖ Xác định thuộc tính là xác định các tính chất đặc trưng của đối tượng (tính chất tạo nên đối tượng)
- ❖ Ví dụ



Ví dụ (tiếp)

Lớp người



Họ và tên:
Ngày sinh:
Giới tính:
Địa chỉ:
Số CMT:

Đối tượng 1



Họ và tên: Nguyễn Thu
Hiền
Ngày sinh: 2/3/1987
Giới tính: Nữ
Địa chỉ: 3 Phố Huế, Hà Nội
Số CMT: 012433763

Đối tượng 2



Họ và tên: Ngô Văn Nam
Ngày sinh: 4/5/1987
Giới tính: Nam
Địa chỉ: Cổ Loa, Đông Anh,
Hà Nội
Số CMT: 012134088

❖ Xây dựng lớp công dân:

- Họ tên
- Ngày sinh
- Giới tính
- Địa chỉ
- Số CMT

❖ Xây dựng phương thức:

- Nhập
- Hiện thị
- ...

? Xác định thuộc tính của bài toán sau (5')

❖ Nhập vào danh sách sinh viên gồm n sinh viên với những thông tin:

- Họ và tên
- Ngày sinh
- Giới tính
- Địa chỉ
- Lớp



hiển thị thông tin theo ngày sinh tăng dần.

❖ Phân tích bài toán sử dụng phương pháp trừu tượng hóa dữ liệu cho biết bài toán gồm những đối tượng nào, thuộc tính và hành vi của chúng.

b. Xác định thành phần phương thức

- ❖ Xác định các hành động trên đối tượng (hành động tác động lên đối tượng và hành động phát sinh bởi đối tượng)
- ❖ Ví dụ
 - Đối tượng phân số có thể có các hành động:
 - Tạo lập phân số với tử số, mẫu số xác định
 - Cộng, trừ, nhân, chia, nghịch đảo, phủ định, rút gọn...phân số.
 - Đối tượng Người có thể có các hành động:
 - Thiết lập thông tin cá nhân, Hiển thị thông tin cá nhân.
 - Tìm kiếm thông tin cá nhân, Thống kê thông tin...

Luyện tập (10')

- ❖ Người ta cần quản lý thông tin về thành phố của các quốc gia. Mỗi quốc gia người ta cần quản lý các thông tin: tên quốc gia, vị trí địa lý, tên thủ đô, số thành phố trực thuộc quốc gia. Với mỗi thành phố người ta quản lý các thông tin như: tên thành phố, vị trí địa lý, số dân. Hệ thống cho phép nhập vào thông tin quốc gia, thông tin thành phố và cũng cho phép hiển thị các thông tin khi cần thiết, tìm kiếm tên thủ đô của quốc gia.
- ❖ Hãy phân tích bài toán và cho biết các lớp đối tượng cần xây dựng cùng thuộc tính và phương thức của chúng.



2.1.3. Định nghĩa và sử dụng lớp

- ❖ Định nghĩa lớp là quá trình khai báo và xây dựng lớp với ngôn ngữ lập trình cụ thể và cài đặt vào máy tính.
- ❖ Định nghĩa lớp với cú pháp C++

```
class tên_lớp
{
    // Khai báo các thành phần dữ liệu (thuộc tính)
    // Khai báo các phương thức (hành vi)
};
// Định nghĩa (xây dựng) các phương thức
```

Định nghĩa các thuộc tính

- ❖ Các thuộc tính thông thường là các thành phần dữ liệu đơn giản như: số nguyên, số thực, kí tự, chuỗi, mảng. Khi đó việc định nghĩa các thuộc tính chính là khai báo các biến thuộc tính.
- ❖ Trong trường hợp các thành phần dữ liệu phức tạp: có thể là kiểu_trừu_tượng_X thì việc trước tiên cần định nghĩa kiểu này trước (Vấn đề này chúng ta sẽ nghiên cứu kỹ hơn ở mục 2.8. Lớp Bao)

Ví dụ

```
class Phanso
{
    int tuso; // tử số
    int mauso; // mẫu số
public:
    void nhapps(); // Phương thức để nhập giá trị cho thành phần dữ liệu
    void hienthi(); // Phương thức để hiển thị giá trị các thành phần dữ
    liệu
};
```

Ví dụ

```
class Nguoi
{
    char hoten[35]; // họ và tên
    char ngaysinh[10]; // ngày tháng năm sinh
    int gioitinh; // giới tính
    char diachi[100]; // địa chỉ
    char email[40]; // thư điện tử
    char CMT[10]; // số chứng minh thư
    public:
    void nhaptt(); // Phương thức để nhập giá trị cho thành phần dữ liệu
    void hienthi(); // Phương thức để hiển thị giá trị các thành phần dữ liệu
};
```

Ví dụ

```
class Diem
{
    double x; //hoành độ
    double y; //tung độ
    public:
        void nhapd(); //Phương thức để nhập giá trị cho thành phần hoành độ
        và tung độ
        void hienthi(); //Phương thức để hiển thị giá trị hoành độ, tung độ của
        điểm
        float KC2D(Diem P); //Phương thức để tính khoảng cách giữa hai điểm
        trên trục tọa độ xOy
};
```

Định nghĩa phương thức

- ❖ Định nghĩa phương thức gồm khai báo và xây dựng, hầu hết các ngôn ngữ lập trình hướng đối tượng việc khai báo và xây dựng gộp làm 1 và thực hiện bên trong định nghĩa lớp như sau:

Class Tên_lớp

```
{  
    //Khai báo thuộc tính  
    //Khai báo và xây dựng phương thức  
    void[Kieu_Du_Lieu] Tên_phương_thức ([Tham đối])  
    {  
        //nội dung phương thức  
    }  
}
```

Định nghĩa phương thức với C++

- ❖ Cú pháp định nghĩa phương thức:

```
void| Kiểu_dữ_liệu Tên_phương_thức ([Tham_sô])  
{  
    //Nội dung phương thức  
    [return;]  
    [return giá_trị;]  
}
```

- ❖ Đối với C++ việc xây dựng phương thức có thể bên trong hoặc bên ngoài lớp
- ❖ Xây dựng bên ngoài lớp theo cú pháp sau:

```
void| Kiểu_dữ_liệu Tên_lớp::Tên_phương_thức ([Tham_sô])  
{  
    //Nội dung phương thức  
    [return;]  
    [return giá_trị;]  
}
```

Ví dụ

```
class Diem
{
    double x; //hoành độ
    double y; //tung độ
public:
    //Khai báo và xây dựng nội dung phương thức nhập điểm bên trong
    lớp Diem
    void nhapd()
    {
        cout<< "\n Nhập vào hoành đo của diem:";
        cin>>x;
        cout<< "\n Nhập vào tung đo của diem:";
        cin>>y;
    }
    //Khai báo phương thức hiển thị Diem
    void hienthi();
};
//Xây dựng nội dung phương thức hiển thị Diem bên ngoài lớp
void Diem::hienthi()
{
    cout<< "("<<x<< ","<<y<< ")";
}

void main()
{
    Diem A;
    A.nhapd();
    cout<< "\n Toa đo diem A:";
    A.hienthi();
    getch();
}
```

Ví dụ

```
class Phanso
{
    int tuso;//tử số
    int mauso;//mẫu số
public:
    void nhapps();//Phương thức để nhập giá trị cho thành phần dữ liệu
    void hienthi();//Phương thức để hiển thị giá trị các thành phần dữ liệu
    Phanso Cong(Phanso P);//Phương thức cộng để cộng hai phân số
};

void Phanso::nhapps()
{
    cout<< "\n Nhập tử số:";cin>>tuso;
    cout<< "\n Nhập mẫu số:";cin>>mauso;
}

void Phanso::hienthi()
{
    cout<<tuso<< "/"<<mauso;
}
```

Ví dụ

```
Phanso Phanso::Cong(Phanso P)
{
    Phanso K;
    K.tuso=tuso*P.mauso+P.tuso*mauso;
    K.mauso=mauso*P.mauso;
    return K;
}

void main()
{
    Phanso A, B, C;
    cout<< "\n Nhap phan so A:";A.nhapps();
    cout<< "\n Nhap phan so B:";B.nhapps();
    C=A.Cong(B);
    cout<< "\n Phan so tong la:";C.hienthi();
    getch();
}
```

❖ Sử dụng lớp:

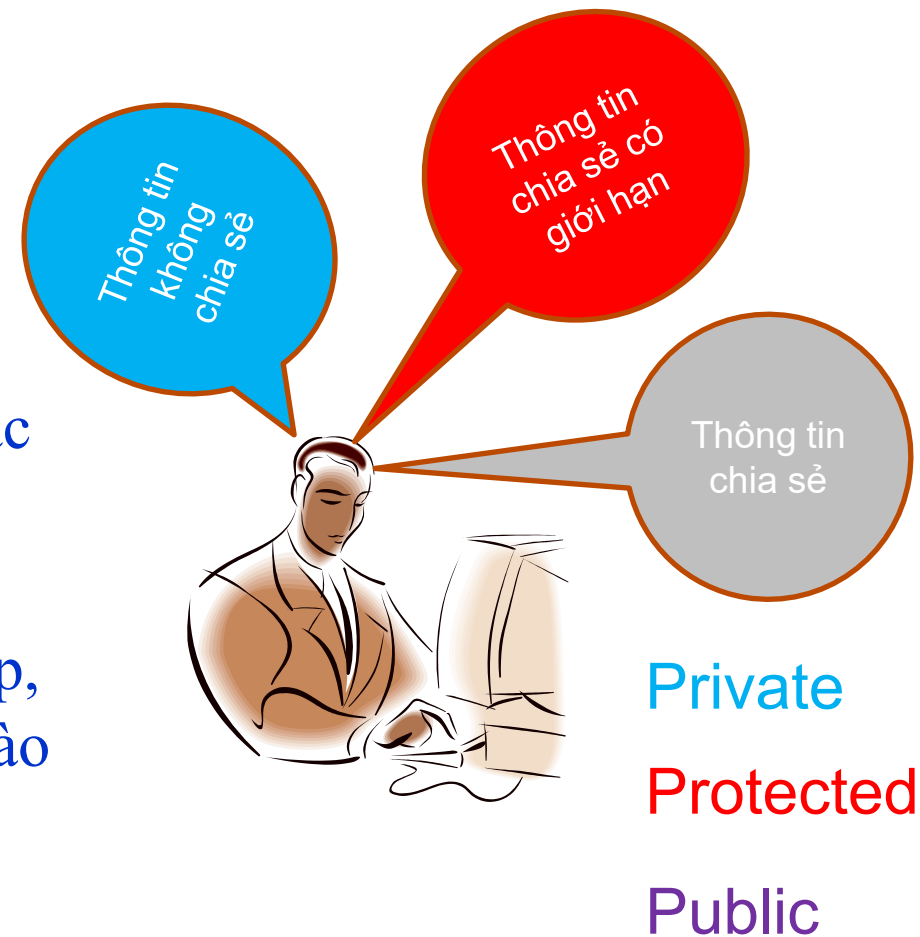
- Khai báo các đối tượng, mảng đối tượng, con trỏ đối tượng
- Thực hiện các thông điệp cần thiết để giải quyết các bài toán đặt ra.

Ví dụ

```
class Hìnhchunhat
{
    private://khai báo phạm vi private
    double cd;//chiều dài hình chữ nhật
    double cr;//chiều rộng hình chữ nhật
    public:
    void nhaptt();//Phương thức để nhập giá trị cho thành phần dữ liệu
    void hienthi();//Phương thức để hiển thị giá trị các thành phần dữ liệu
    double dientich()
    {
        return cd*cr; //hoàn toàn có thể sử dụng thành phần cd và cr ở đây
    }
};
void main()
{
    Hìnhchunhat h1;//h1 là một đối tượng của lớp hìnhchunhat
    h1.cd=7; //câu lệnh lỗi không được phép truy cập thành phần cd vì thuộc phạm vi private
    h1.cr=5; //câu lệnh lỗi không được phép truy cập thành phần cr vì thuộc phạm vi private
    ...
}
```

2.2. Phạm vi Private, Protected, Public

- ❖ Tính che dấu thông tin trong ngôn ngữ lập trình hướng đối tượng có được là nhờ sự phạm vi private, protected, public khi định nghĩa lớp đối tượng.
- ❖ Private: các thành phần được khai báo sau từ khóa private chỉ được sử dụng bên trong lớp (các phương thức bên trong lớp có thể sử dụng).
- ❖ Khai báo thành phần dữ liệu là private để bảo đảm tính giấu kín, bảo vệ an toàn dữ liệu của lớp, không cho phép các hàm bên ngoài xâm nhập vào dữ liệu của lớp.



Ví dụ tác dụng phạm vi private

```
class Hìnhchunhat
{
    private://khai báo phạm vi private
    double cd;//chiều dài hình chữ nhật
    double cr;//chiều rộng hình chữ nhật
    public:
    void nhaptt();//Phương thức để nhập giá trị cho thành phần dữ liệu
    void hienthi();//Phương thức để hiển thị giá trị các thành phần dữ liệu
    double dientich()
    {
        return cd*cr; //hoàn toàn có thể sử dụng thành phần cd và cr ở đây
    }
};
void main()
{
    Hìnhchunhat h1; //h1 là một đối tượng của lớp hìnhchunhat
    h1.cd=7; //câu lệnh lỗi không được phép truy cập thành phần cd vì thuộc phạm vi private
    h1.cr=5; //câu lệnh lỗi không được phép truy cập thành phần cr vì thuộc phạm vi private
    ...
}
```

2.2. Phạm vi Private, Protected, Public

- ❖ **Public:** các thành phần public (công cộng) được phép sử dụng ở cả bên trong và bên ngoài lớp.

```
class Hìnhchunhat
{
    private://khai báo phạm vi private
    double cd;//chiều dài hình chữ nhật
    double cr;//chiều rộng hình chữ nhật
    public:
    void nhaptt();//Phương thức để nhập giá trị cho thành phần dữ liệu
    void hienthi();//Phương thức để hiển thị giá trị các thành phần dữ liệu
    double dientich()
    {
        return cd*cr;
    }
};
void main()
{
    Hìnhchunhat h1; //h1 là một đối tượng của lớp hìnhchunhat
    h1.dientich(); //gọi tới phương thức tính diện tích hoàn toàn có thể
    //được thực hiện vì dientich() thuộc phạm vi public của lớp.
    ...
}
```

2.2. Phạm vi Private, Protected, Public

- ❖ Các phương thức thường khai báo là public để chúng có thể được gọi tới (sử dụng) từ các hàm khác trong chương trình.
- ❖ **Protected:** các thành phần protected được phép sử dụng ở bên trong lớp và lớp con của lớp mà nó cho phép kế thừa. Chúng ta sẽ tìm hiểu kỹ hơn về sự hoạt động của phạm vi này trong chương 3.

Phạm vi thông tin và cách phân chia

- ❖ Thông tin a: có những thông tin (điều) ta không nói với ai bao giờ khi đó thông tin này thuộc phạm vi private
- ❖ Thông tin b: như công thức nấu rượu gia truyền, làm bánh gia truyền- chỉ truyền cho con cháu khi đó thông tin này thuộc phạm vi protected
- ❖ Thông tin c: các thông tin trao đổi với nhau hàng ngày (giá vàng, phim ảnh....) thông tin thuộc phạm vi public

2.3. Đối tượng, mạng đối tượng.

2.3.1. Khái niệm.

2.3.2. Khai báo và sử dụng đối tượng.

Đối tượng



Mạng đối tượng



2.3.2. Khai báo và sử dụng đối tượng

- ❖ Đối tượng được xét đến ở đây là đối tượng trong máy tính (sản sinh do lớp đối tượng).

Đối tượng= biến đối tượng

- ❖ Với mỗi khai báo:

Lớp_đối_tượng đối_tượng;

Hay

Kiểu_dữ_liệu_trừu_tượng biến;

Cho ta các đối tượng trong máy tính

Ví dụ: Phan_so A,B;

Chien_binh C1, C2;

Truy cập thành viên lớp thông qua đối tượng

- ❖ Các đối tượng tồn tại như đã nghiên cứu gồm thuộc tính và phương thức (đã được xây dựng). Để truy cập vào các thành phần này ta truy cập thông qua cú pháp:

Tên_đối_tượng. Tên_thành_phần

- ❖ Thành phần có thể là thuộc tính hoặc phương thức.
- ❖ Lưu ý: Lời gọi truy cập này có phạm vi phụ thuộc phạm vi của thành_phần được gọi.

Ví dụ

```
class lopA
{
    private:
        int x;
        int y;
    public:
        float z;
        float t;
        ...
};

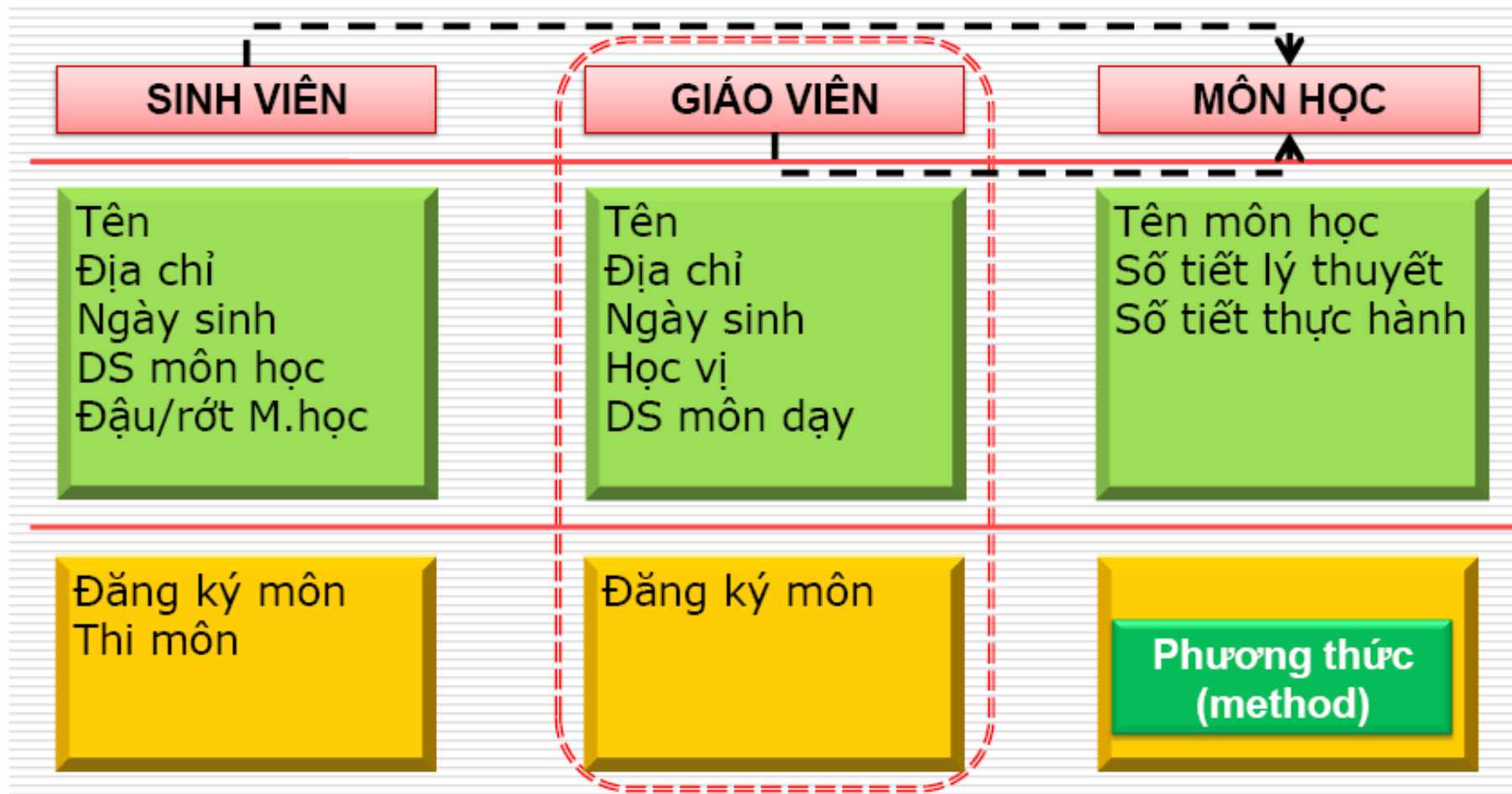
void main()
{
    lopA d1,d2;
    d1.z=10.8; //truy cập vào thuộc tính z của đối tượng d1
    d1.t=-9.3; //truy cập vào thuộc tính t của đối tượng d1
    d2.t=12.8; //truy cập vào thuộc tính t của đối tượng d2
}
```

2.3.3. Mảng đối tượng

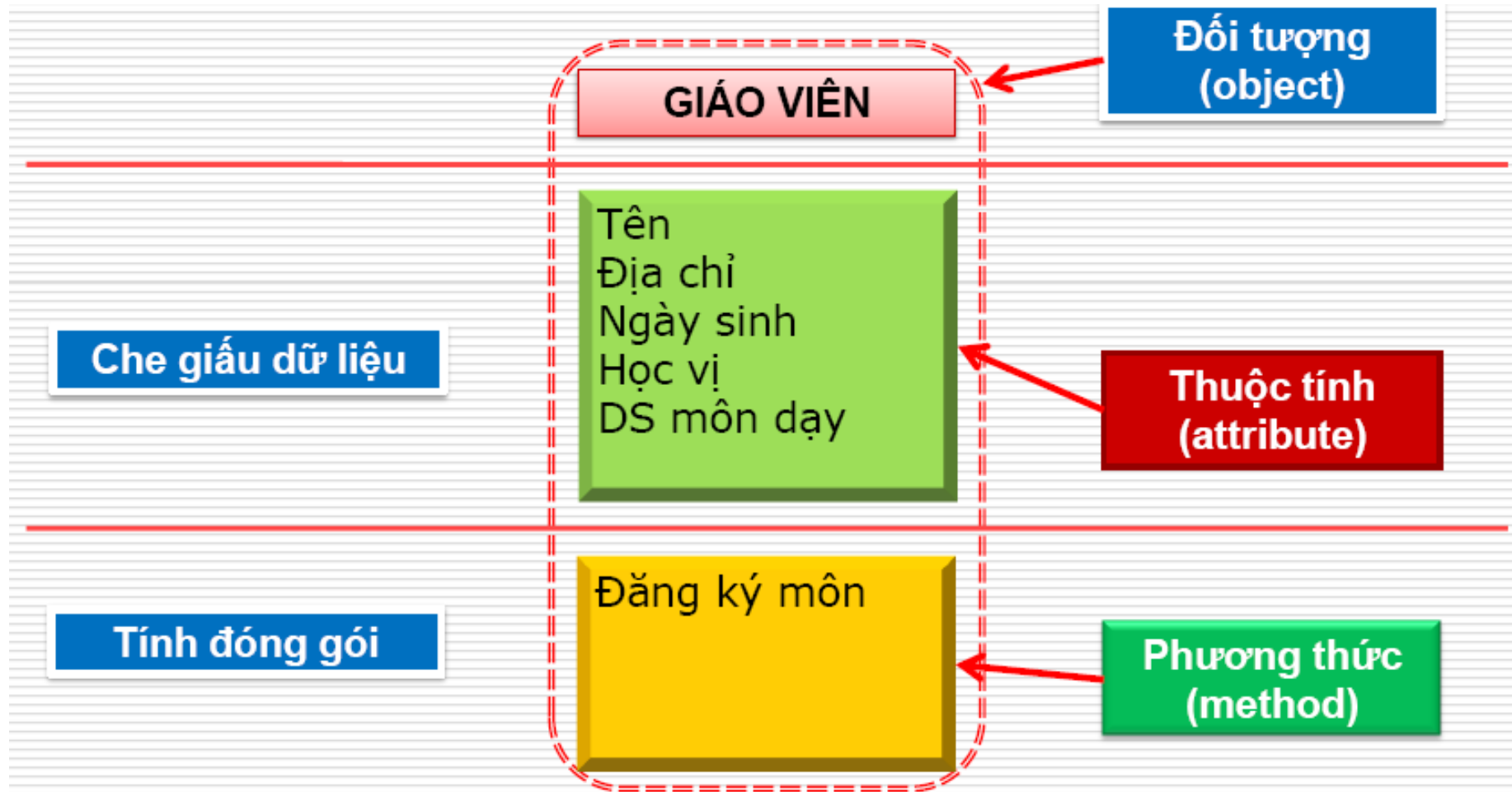
- ❖ Giống như các kiểu dữ liệu thông thường, kiểu dữ liệu trừu tượng khi sinh ra trong máy tính có thể khai báo mảng (danh sách) các đối tượng.
- ❖ Việc truy cập thành phần của lớp thông qua mảng đối tượng như sau:

Tên_mảng[chỉ số].Thành_phần

Thảo luận 15'



Thảo luận 15'



2.4. Hàm tạo và hàm hủy.

2.4.1. Hàm tạo

- a. Khái niệm và lý do cần xây dựng
- b. Xây dựng phương thức khởi tạo
- c. Lời gọi

2.4.2. Hàm hủy

- a. Khái niệm và lý do cần xây dựng
- b. Xây dựng phương thức hủy bỏ
- c. Lời gọi

2.4.1. hàm tạo

a. Khái niệm và lý do cần xây dựng

- ❖ Là một phương thức đặc biệt của lớp dùng để khởi gán các giá trị cho các thuộc tính của đối tượng và cũng có thể cấp phát bộ nhớ cho thành phần thuộc tính kiểu con trỏ của lớp. Hàm tạo được khai báo bên trong lớp và có thể xây dựng bên trong hoặc bên ngoài lớp.
- ❖ Xây dựng hàm tạo nhằm khởi gán giá trị ban đầu cho đối tượng, cấp phát bộ nhớ cho đối tượng

2.4.1. hàm tạo

b. Xây dựng phương thức khởi tạo

```
Tên_lớp([Tham số])  
{  
    //Nội dung hàm tạo  
}
```

Đặc điểm chung của phương thức khởi tạo

- Thông thường hàm tạo có tên trùng tên lớp (C++, C#)
- Không bắt đầu là từ khóa Void hay kiểu dữ liệu
- Không trả về dữ liệu

Ví dụ

```
class Diem
{
    double x; //hoành độ
    double y;//tung độ
public:
    //Khai báo và xây dựng hàm tạo của lớp Diem
    Diem() //Hàm tạo không đối
    {
        //Khởi tạo ở gốc tọa độ
        x=0.0; y=0.0;
    }
    void nhapd();
    void hienthi();
};
```

Ví dụ

```
class Diem
{
    double x; //hoành độ
    double y;//tung độ
public:
    //Khai hàm tạo của lớp Diem
    Diem();
    void nhapd();
    void hienthi();
};
//Xây dựng hàm tạo bên ngoài lớp
Diem:: Diem() //Hàm tạo không đối
{
    //Khởi tạo ở gốc tọa độ
    x=0.0; y=0.0;
}
```

2.4.1. Hàm tạo

c. Lời gọi

- ❖ Phương thức khởi tạo không cần được triệu gọi giống phương thức thông thường
- ❖ Nó sẽ tự động được gọi khi khai báo đối tượng.

2.4.1. Hàm tạo

d. Phân loại

- Hàm tạo không đối
 - Hàm tạo có đối
 - Hàm tạo sao chép
- ❖ Cú pháp các loại hàm tạo với C++

```
Tên_lớp ()  
{  
    //Nội dung hàm tạo  
}
```

2.4.1. Hàm tạo

❖ Cú pháp hàm tạo có đối và sao chép

```
Tên_lớp (tham số 1[, tham số 2, ...tham số i] )  
{  
    //Nội dung hàm tạo  
}
```

```
Tên_lớp (const Tên_lớp &đối_tượng)  
{  
    //Nội dung hàm tạo  
}
```

Ví dụ: các loại hàm tạo

```
class Diem
{
    double x; //hoành độ
    double y; //tung độ
public:
    //Khai hàm tạo của lớp Diem
    Diem(); //hàm tạo không tham đối
    Diem(double x1, double y1); //hàm tạo có đối
    Diem(const Diem &M); //hàm tạo sao chép
    void hienthi()
    {
        cout<< "("<<x<< ","<<y<< ")";
    }
};
//Xây dựng hàm tạo bên ngoài lớp
Diem:: Diem() //Hàm tạo không đối
{
    //Khởi tạo ở gốc tọa độ
    x=0.0; y=0.0;
}
Diem:: Diem(double x1, double y1) //Hàm tạo có đối
{
    x=x1; y=y1;
}
```

Ví dụ (tiếp)

```
Diem:: Diem(const Diem & M) //Hàm tạo sao chép
{
    x=M.x; y=M.y;
}
void main()
```

```
{
    Diem A;//gọi tới hàm tạo không đối
    Diem B(6.5, 3.7);//gọi tới hàm tạo có đối
    Diem C(B);//gọi tới hàm tạo sao chép
    cout<<"\n Toa do diem A:";A.hienthi(); // Tọa độ điểm A (0.0,0.0)
    cout<<"\n Toa do diem B:"; B.hienthi(); //Tọa độ điểm B (6.5, 3.7)
    cout<<"\n Toa do diem C:";C.hienthi(); //Tọa độ điểm C (6.5, 3.7)
    getch();
}
```

Vấn đề Hàm tạo mặc định



Đoạn chương trình mô tả hàm tạo

```
class Diem
{
    double x; //hoành độ
    double y; //tung độ
public:
    //Khai hàm tạo của lớp Diem
    Diem(double x1, double y1); //hàm tạo có đối
    void hienthi()
    {
        cout<< "("<<x<< ","<<y<< ")";
    }
};
//Xây dựng hàm tạo bên ngoài lớp
Diem:: Diem(double x1, double y1) //Hàm tạo có đối
{
    x=x1; y=y1;
}
void main()
{
    Diem B(6.5, 3.7); //gọi tới hàm tạo có đối
    Diem A; //gọi tới hàm tạo không đối và bị lỗi vì chương trình không
    xây dựng hàm tạo không đối
```

2.4.2. Hàm hủy

a. Khái niệm và lý do cần xây dựng

- ❖ Hàm hủy cũng là một phương thức của lớp nhưng phương thức này khá đặc biệt nó có nhiệm vụ hủy đi bộ nhớ được cấp phát bởi hàm tạo.
- ❖ Hàm hủy sẽ được gọi trước khi kết thúc hàm hoặc trước khi chương trình ra khỏi phạm vi của đối tượng.
- ❖ Nếu một lớp không định nghĩa hàm hủy thì chương trình dịch sẽ tự phát sinh một hàm hủy mà không làm gì cả.

b. Xây dựng phương thức hủy bỏ

- ❖ Cú pháp xây dựng hàm hủy với C++

```
~Tên_lớp ()  
{  
    //Nội dung hàm hủy  
}
```

Ví dụ

```
class Diem
{
    double *x; //hoành độ
    double *y; //tung độ
public:
    //Khai hàm tạo của lớp Diem
    Diem(); //hàm tạo không tham đối
    Diem(double x1, double y1); //hàm tạo có đối
    Diem(const Diem &M); //hàm tạo sao chép
    void hienthi()
    {
        cout<< "("<<*x<< ","<<*y<< ")";
    }
    //Khai báo và xây dựng hàm hủy
    ~Diem()
    {
        delete x;
        delete y;
    }
};
```

Ví dụ

```
class Diem
{
    double *x; //hoành độ
    double *y; //tung độ
public:
    //Khai hàm tạo của lớp Diem
    Diem(); //hàm tạo không tham đối
    Diem(double x1, double y1); //hàm tạo có đối
    Diem(const Diem &M); //hàm tạo sao chép
    void hienthi()
    {
        cout<< "("<<*x<< ","<<*y<< ")";
    }
    //Khai báo và xây dựng hàm hủy
    ~Diem()
    {
        delete x;
        delete y;
    }
};
```

Vi dụ

```
//Xây dựng hàm tạo bên ngoài lớp
Diem:: Diem() //Hàm tạo không đối
{
    //Cấp phát bộ nhớ
    x=new double;
    y=new double;
    //Khởi gán giá trị ban đầu
    *x=0.0; *y=0.0;
}
Diem:: Diem(double x1, double y1) //Hàm tạo có đối
{
    //Cấp phát bộ nhớ
    x=new double;
    y=new double;
    //Gán giá trị
    *x=x1; *y=y1
}
```

Ví dụ

```
Diem:: Diem(const Diem & M) //Hàm tạo sao chép
{
    //Cấp phát bộ nhớ
    x=new double;
    y=new double;
    //Gán giá trị
    *x=*M.x; *y=*M.y;
}
void main()
{
    Diem A;//gọi tới hàm tạo không đối
    Diem B(6.5, 3.7);//gọi tới hàm tạo có đối
    Diem C(B);//gọi tới hàm tạo sao chép
    cout<<"\n Toa do diem A:";A.hienthi(); // Tọa độ điểm A (0.0,0.0)
    cout<<"\n Toa do diem B:"; B.hienthi(); //Tọa độ điểm B (6.5, 3.7)
    cout<<"\n Toa do diem C:";C.hienthi(); //Tọa độ điểm C (6.5, 3.7)
    getch();
}
```

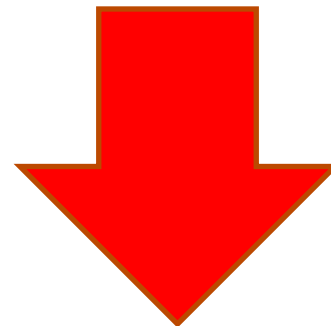
2.5. Con trỏ this

- ❖ Mỗi lớp khi khai báo sẽ tồn tại một con trỏ this đặc biệt trỏ tới các thành phần của lớp. Nó sẽ quản lý các thành phần của lớp. Như vậy khi xây dựng lớp các thành phần của lớp được quản lý thông qua con trỏ this trỏ đến chính nó.

Ví dụ như sau:

```
class A
{
    int x;
    void TT()
    {
        x=10;//là tương đương this->x=10;
    }
};
```

this



2.6. Lớp bao

- ❖ Lớp bao là lớp mà thành viên thuộc tính của nó là đối tượng của lớp khác.

Ví dụ:

```
class A;
```

```
class B;
```

```
class C
```

```
{
```

```
    A a;
```

```
    B b,c;
```

```
    float d;
```

```
    ...
```

```
};
```

- ❖ Khi đó C được gọi là lớp bao của lớp A và lớp B. Lớp A, B được gọi là lớp thành viên của C.



Vấn đề sử dụng hàm tạo của lớp thành viên trong lớp bao

- ❖ Việc gọi tới các thuộc tính và phương thức của đối tượng làm thành viên của lớp bao được thực hiện theo nguyên tắc như gọi từ chương trình chính. Tuy nhiên khi xây dựng hàm tạo cho lớp bao ta phải sử dụng hàm tạo của lớp thành viên để khởi gán giá trị cho các đối tượng thành viên.
- ❖ Cú pháp xây dựng hàm tạo lớp bao như sau:

Hàm tạo lớp bao với C++

```
Tên_lớp_bao([Tham_sô] ): Dt1([Tham_sô])[, Dt2([Tham_sô])...,  
Dtn([Tham_sô])]  
{  
    //Câu lệnh khởi gán giá trị cho các thành viên thường của lớp bao  
}
```

Ví dụ: Xây dựng lớp tam giác tạo bởi 3 điểm

```
class Diem
{
    private:
        int x,y;
    public:
        Diem();
        Diem(int x,int y);
        Diem (const Diem &D);
        friend ostream&operator<<(ostream &os,Diem D);
        friend istream&operator>>(istream &is,Diem &D);
        float KC2D(Diem D);
};

Diem::Diem()
{
    x=0;y=0;
}

Diem::Diem(int x,int y)
{
    this->x=x;this->y=y;
}

Diem::Diem(const Diem &D)
{
    x=D.x;y=D.y;
}

ostream&operator<<(ostream&os,Diem D)
{
    os<<"("<<D.x<<","<<D.y<<")";
    return os;
}
```

Ví dụ: Xây dựng lớp tam giác tạo bởi 3 điểm

```
istream&operator>>(istream&is, Diem &D)
{
    cout<<"\n Nhập toa do x:";is>>D.x;
    cout<<"\n Nhập toa do y:";is>>D.y;
    return is;
}
float Diem::KC2D(Diem D)
```

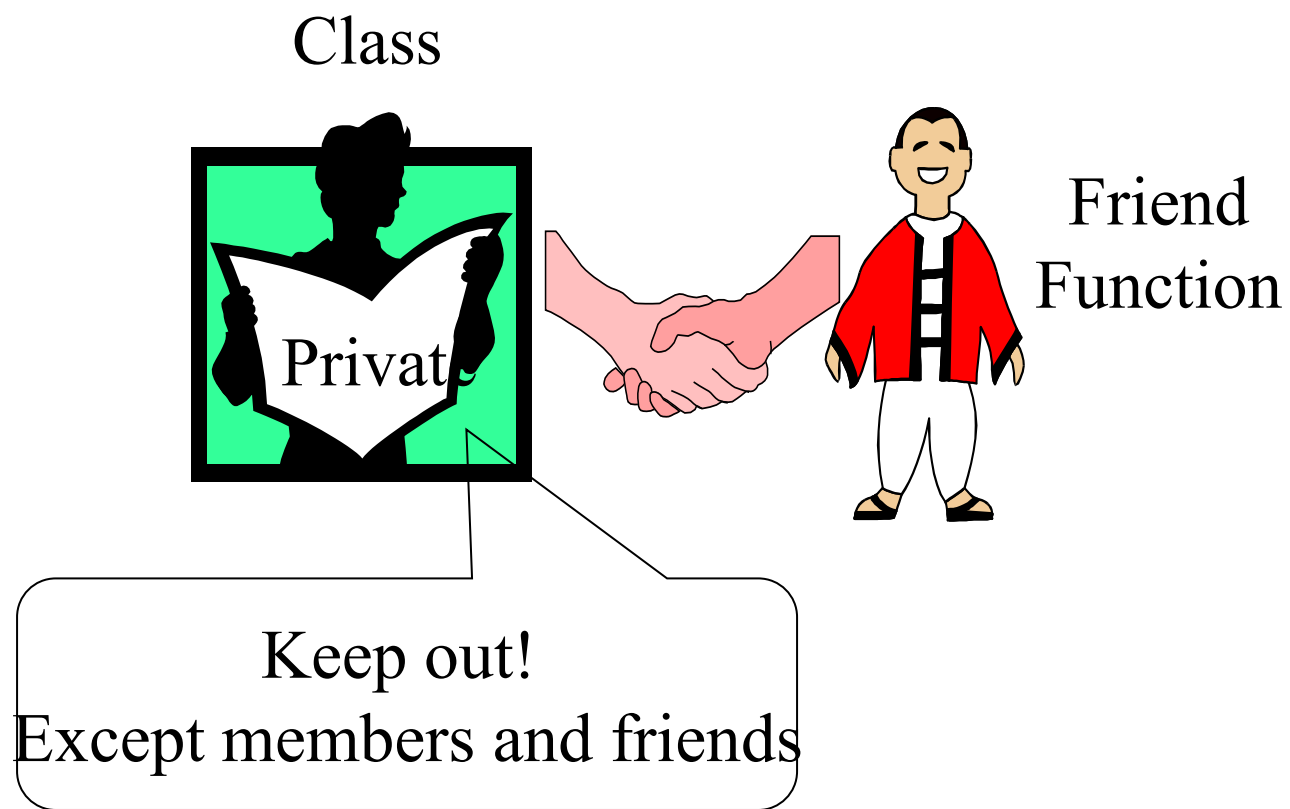
```
{
    return sqrt((x-D.x)*(x-D.x)+(y-D.y)*(y-D.y));
}
//*****TG
class Tag
{
    private:
        Diem A,B,C;
    public:
        Tag(Diem X, Diem Y, Diem Z):A(X),B(Y),C(Z){} //sử dụng hàm tạo
        của lớp thành viên trong lớp bao
        Tag(const Tag&T)
        {
            A=T.A;
            B=T.B;
            C=T.C;
        }
}
```

Ví dụ: Xây dựng lớp tam giác tạo bởi 3 điểm

```
        return sqrt(p*(p-a)*(p-b)*(p-c));  
    }  
float Tag::Cv()  
{  
    float a,b,c;  
    a=B.KC2D(C);  
    b=A.KC2D(C);  
    c=A.KC2D(B);  
    return (a+b+c);  
}  
void main()  
{  
    clrscr();  
    Diem A(0,3),B(0,0),C(4,0);  
    Tag T(A,B,C);  
    Tag P(T);  
    cout<<"P:"<<P;  
    cout<<"\n Nhap tam giac T:"<<cin>>T;  
    cout<<"\n Dien tich T la:"<<T.Dt();  
    cout<<"\n Chu vi T la:"<<T.Cv();  
    getch();  
}
```

2.7. Hàm bạn

- a. Khái niệm
- b. Khai báo và xây dựng, cách sử dụng



Hàm bạn

- ❖ Hàm bạn của một lớp không phải là phương thức của lớp nhưng có thể truy cập đến thành phần riêng của lớp (bất kì thành phần nào của lớp). Một hàm có thể là bạn của nhiều lớp
- ❖ Khai báo và xây dựng hàm bạn với C++
Hàm bạn phải được khai báo bên trong lớp và xây dựng bên ngoài lớp

Cú pháp khai báo và xây dựng

```
class lopX
{
    ...
    //Khai báo hàm bạn
    friend void|Kiểu_dữ_liệu ([Tham_số]);
    ...
};
...
//Xây dựng hàm bạn
void|Kiểu_dữ_liệu ([Tham_số])
{
    //nội dung hàm
}
```

Ví dụ

```
class Diem
{
    double x; //hoành độ
    double y; //tung độ
public:
    //Khai báo hàm tạo của lớp Diem
    Diem()
    {
        x=0.0; y=0.0;
    }
    Diem(double x1, double y1)
    {
        x=x1; y=y1;
    }
    Diem(const Diem &M)
    {
        x=M.x; y=M.y;
    }
    void hienthi()
    {
        cout<< "("<<x<< ","<<y<< ")";
    }
}
```

```
double KC2D(Diem P) //tính khoảng cách hai điểm trên trục tọa độ
{
    return sqrt((x-P.x)*(x-P.x)+(y-P.y)*(y-P.y));
}
//Khai báo hàm bạn kiểm tra xem hai điểm có trùng nhau không
friend int kiểmtra(Diem A, Diem B);
};
//Xây dựng hàm bạn bên ngoài lớp
int kiểmtra(Diem A, Diem B)
{ //truy cập được thành phần riêng x,y của lớp Diem
    if (A.x==B.x) && (A.y==B.y))
        return 1;
    else
        return 0;
}
//Hàm thông thường
double dientich(Diem A, Diem B, Diem C)
{
    double a,b,c,p, s;
    s=0;
    a=B.KC2D(C);
```

```
b=C.KC2D(A);
c=A.KC2D(B);
if (kiemtra(A,B)==0 && kiemtra(B,C)==0 && kiemtra(A,C)==0)
{
    p=(a+b+c)/2;
    s=sqrt(p*(p-a)*(p-b)*(p-c));
}
return s;
}
void main()
{
    Diem A(0,0),B(3,0), C(0,4);
    cout<<"\n Toa do diem A:";A.hienthi(); // Tọa độ điểm A (0.0, 0.0)
    cout<<"\n Toa do diem B:"; B.hienthi(); //Tọa độ điểm B (3.0, 0.0)
    cout<<"\n Toa do diem C:";C.hienthi(); //Tọa độ điểm C (0.0, 4.0)
    double s=dientich(A,B,C);
    if (s==0)
        cout<<"A, B, C không là tam giác";
    else
        cout<<"\n Dien tich tam giac la:"<<s; //S=12
    getch();
}
```

Hàm là bạn của nhiều lớp

Một hàm có thể làm bạn của nhiều lớp. Khi đó hàm có thể truy cập vào thành viên của tất cả các lớp nó coi là bạn



```
class A
{
    int x;
    friend void F(A,B);
};
class B
{
    int y;
    friend void F(A,B);
};
```

Phương thức của một lớp là bạn của lớp khác

❖ Xét ví dụ:

```
class A  
{  
    int x;  
};  
class B
```

```
{  
    int y;  
    int F(A a);  
};
```

```
class B;  
class A  
{  
    int x;  
    friend int B::F(A a);  
};  
class B  
{  
    int y;  
    int F(A a);  
};
```

Hàm bạn, hàm thông thường, phương thức của lớp



2.8. Lớp bạn

- ❖ **Định nghĩa :** Nếu lớp A được khai báo là bạn của lớp B thì tất cả các phương thức của lớp A đều có thể truy cập đến các thành phần riêng của lớp B. Lớp A là bạn của lớp B nhưng chưa chắc B đã là bạn của A. Một lớp có thể bạn của nhiều lớp.

```
class B;  
class A;  
{  
...  
//Khái báo lớp B là bạn của lớp A  
friend class B;  
...  
};
```

Ví dụ về lớp
bạn



2.9. Khả năng nạp chồng

- ❖ 2.9.1. Nạp chồng hàm
- ❖ 2.9.2. Nạp chồng toán tử
- ❖ 2.9.3. Nạp chồng phương thức



Chồng hàm

- ❖ **Khái niệm:** là khả năng các hàm trùng tên nhau nhưng khác nhau ít nhất một trong 3 tiêu chí sau:
 - Kiểu dữ liệu trả về của hàm
 - Kiểu dữ liệu của tham số truyền vào cho hàm
 - Số lượng tham số truyền vào của hàm.
- ❖ **Ví dụ 1:**
 - `int trituyetdoi(int a);` *//trị tuyệt đối số nguyên*
 - `long trituyetdoi (long a);` *//tính trị tuyệt đối số nguyên dài*
 - `double trituyetdoi(double a);` *//tính trị tuyệt đối số thực dài*
- ❖ **Ví dụ 2:**
 - `void hoanvi(int &a, int &b);` *//hoán vị hai số nguyên*
 - `void hoanvi(char &a, char &b);` *//hoán vị hai kí tự*
 - `void hoanvi(float &a, float &b);` *//hoán vị hai số thực*

Chồng toán tử

- ❖ Giống như hàm các toán tử cũng có khả năng nạp chồng nhau. Chẳng hạn như đối với phép toán cộng:

$$2 + 6 = 8$$

$$2.7 + 8.2 = 10.9$$

- ❖ Cú pháp:

```
void|Kiểu_dữ_liệu operator Toán _tử ([Tham_sô])  
{  
    //Nội dung  
}
```

Ví dụ: Xây dựng toán tử + cho lớp phân số

```
class Phanso
{
    int tuso; //tử số
    int mauso; //mẫu số
public:
    void nhapps();
    void hienthi();
    //Khai báo chồng toán tử cộng cho lớp phân số
    Phanso operator+(Phanso P);
};

void Phanso::nhapps()
{
    cout<< "\n Nhap tu so:"; cin>>tuso;
    cout<< "\n Nhap mau so:"; cin>>mauso;
}

void Phanso::hienthi()
{
    cout<<tuso<< "/"<<mauso;
```

Ví dụ: Xây dựng toán tử + cho lớp phân số

```
//Xây dựng toán tử cộng cho lớp Phân số
Phanso Phanso::operator+(Phanso P)
{
    Phanso K;
    K.tuso=tuso*P.mauso+P.tuso*mauso;
    K.mauso=mauso*P.mauso;
    return K;
}

void main()
{
    Phanso A, B, C;
    cout<< "\n Nhap phan so A:";A.nhapps();
    cout<< "\n Nhap phan so B:";B.nhapps();
    //Lời gọi toán tử cộng của lớp phân số
    C=A+B;
    cout<< "\n Phan so tong la:";C.hienthi();
    getch();
}
```



- ❖ **Lưu ý:** Ta có thể xây dựng toán tử dưới vai trò là hàm thông thường, hàm bạn của lớp và phương thức của lớp.
- ❖ Định nghĩa toán tử Logic

```
int operator!(Tham_số);
```

Ví dụ: Khai báo toán tử kiểm tra có phải là phân số không

```
class Phanso
{
    int tuso; //tử số
    int mauso; //mẫu số
public:
    void nhapps();
    void hienthi();
    //Khai báo chồng toán tử kiểm tra có phải là phân số không
    int operator!();
};
```

Một số toán tử đặc biệt

❖ Định nghĩa toán tử gán (=)

- Toán tử gán xây dựng cho lớp là toán tử đặc biệt so với các toán tử khác. Nếu trong lớp chưa định nghĩa toán tử gán thì chương trình dịch sẽ tự phát sinh một toán tử gán để thực hiện lệnh gán hai đối tượng cho nhau.
- Ví dụ Phanso A, B(3,4);

A=B;

Khi đó toán tử gán mặc định làm nhiệm vụ sao chép nội dung của đối tượng B vào đối tượng A theo từng bit một.

Toán tử gán

- ❖ Giống với hàm tạo, hàm hủy nếu lớp không có thành viên kiểu con trỏ hay tham chiếu thì toán tử gán mặc định là đủ và không cần xây dựng toán tử gán cho lớp.
- ❖ Nhưng đối với lớp có thành viên thuộc tính kiểu con trỏ như lớp Diem trong ví dụ 2.18 thì hàm tạo mặc định là không đủ. Việc xây dựng toán tử gán là cần thiết.
- ❖ Cú pháp khai báo toán tử gán như sau:

Cách 1

```
void operator=(const Tên_lớp & đối_tượng)
{
    //nội dung
}
```

Cách 2

```
const Tên_lớp operator=(const Tên_lớp & đối_tượng)
{
    //nội dung
}
```

Cú pháp khai báo toán tử gán

- ❖ Với cách khai báo số 1 thì việc thực hiện câu lệnh gán liên tiếp như sau là không được phép

$$u=v=k=h;$$

- ❖ Sinh viên hãy thực hiện xây dựng toán tử gán cho lớp Diem trong ví dụ 2.18. Và viết chương trình chính để gọi tới chúng.

Định nghĩa toán tử tăng (++), giảm (--)

- ❖ Cú pháp khai báo toán tử tăng, giảm cũng giống với khai báo các toán tử khác.
- ❖ Vấn đề gặp phải là toán tử tăng thì có hai loại tăng trước và tăng sau; toán tử giảm cũng có hai loại giảm trước và giảm sau.
- ❖ Cú pháp để phân biệt toán tử tăng trước và tăng sau như sau:

Khai báo toán tử tăng trước

```
void|Kiểu_dữ_liệu operator++();
```

Khai báo toán tử tăng sau

```
void|Kiểu_dữ_liệu operator++(int);
```

Định nghĩa toán tử (>>), (<<)

- ❖ Toán tử nhập >>, xuất << là toán tử đặc biệt nằm trong lớp `iostream.h` vì vậy khi tiến hành nạp chồng toán tử nhập, xuất của lớp thì ta phải xây dựng nguyên tắc hàm bạn của lớp.
- ❖ Cú pháp toán tử nhập như sau:

```
istream &operator>>(istream & is, Kiểu_dữ_liệu &biên)
{
    //nội dung
    return is;
}
```

Ví dụ: Xây dựng toán tử nhập cho lớp phân số

```
class Phanso
{
    int tuso; //tử số
    int mauso; //mẫu số
public:
    //Khai báo hàm bạn toán tử nhập cho lớp phân số
    friend istream &operator>>(istream &is, Phanso &P);
    void hienthi();
};
//Xây dựng toán tử nhập cho lớp phân số
istream &operator>>(istream &is, Phanso &P)
{
    cout<< "\n Nhập tu so:"; is>>P.tuso;
    cout<< "\n Nhập mau so:"; is>>P.mauso;
    return is;
}
void Phanso::hienthi()
{
    cout<<tuso<< "/"<<mauso;
}
void main()
{
    Phanso A, B;
    cout<< "\n Nhập phân số A:"; cin>>A;
    cout<< "\n Nhập phân số B:"; cin>>B;
    getch();
}
```

Định nghĩa toán tử xuất

```
ostream &operator<<(ostream & os, Kiểu_dữ_liệu biên)
{
    //nội dung
    return os;
}
```

Ví dụ: Xây dựng toán tử xuất cho lớp phân số

```
class Phanso
{
    int tuso; // tử số
    int mauso; // mẫu số
public:
    friend istream &operator>>(istream &is, Phanso &P);
    // Khai báo hàm bạn toán tử xuất cho lớp phân số
    friend ostream &operator<<(ostream &os, Phanso P);
};

istream &operator>>(istream &is, Phanso &P)
{
    cout<< "\n Nhập tử số:"; is>>P.tuso;
    cout<< "\n Nhập mẫu số:"; is>>P.mauso;
    return is;
}
// Xây dựng toán tử nhập cho lớp phân số
ostream &operator<<(ostream &os, Phanso P)
{
    os<<P.tuso<< "/"<<P.mauso;
    return os;
}

void main()
{
    Phanso A, B;
    cout<< "\n Nhập phân số A:"; cin>>A;
    cout<< "\n Nhập phân số B:"; cin>>B;
    cout<< "\n Phân số A vừa nhập là:"<<A;
    cout<< "\n Phân số B vừa nhập là:"<<B;
    getch();
}
```

1. Xây dựng lớp Time để lưu trữ giờ, phút, giây

- Xây dựng hàm tạo để khởi tạo giá trị ban đầu cho giờ, phút, giây.
- Xây dựng phương thức hiển thị dữ liệu thời gian ra màn hình.
- Xây dựng phép cộng để cộng giữa dữ liệu thời gian và một số nguyên là số giây
- Xây dựng phép trừ để trừ giữa dữ liệu thời gian
- Viết hàm chính khởi tạo một đối tượng Time; hiển thị giá trị ra màn hình; rồi gọi tới tất cả các phương thức, toán tử đã xây dựng.

2. Xây dựng lớp Nhanvien gồm họ tên, ngày sinh, giới tính, mã nhân viên, Số CMT, lương cơ bản, phụ cấp.
- Xây dựng phương thức nhập, xuất thông tin của nhân viên.
 - Xây dựng hàm so sánh để so sánh tuổi của hai nhân viên trong công ty.
 - Xây dựng phương thức tính lương của nhân viên biết rằng lương=lương cơ bản *3+ phụ cấp.
 - Viết hàm chính khai báo một danh sách nhân viên gồm ít nhất 5 nhân viên. Nhập vào thông tin của nhân viên. Hiển thị ra màn hình danh sách nhân viên đã được sắp xếp theo tuổi giảm dần gồm có cả lương được lĩnh.